

Lista de Comandos Básicos - MongoDB

Nome: Dominique Eduarda Silva de Jesus

Prontuário: PT3036472

- 1) **show dbs:** lista todos os bancos de dados do cluster do MongoDB ao qual estamos conectados.
- 2) **show collections:** lista todas as coleções do banco de dados que estamos utilizando no momento.
- 3) **use <nome-do-banco>:** seleciona o banco com o nome especificado ou, caso este não exista, cria um banco com o nome especificado.
- 4) Criação de variáveis:
 - a) **var <nome-da-variável>:** cria uma variável com escopo de função (existe em todo o programa), a qual é possível redeclarar e reatribuir valor. No hoisting, variáveis declaradas com *var* recebem o valor inicial *undefined*.
NOTA: Não é recomendada;
 - b) **let <nome-da-variável>:** cria uma variável com escopo de bloco (existe apenas no bloco - {} - em que foi criada), a qual é possível redeclarar e reatribuir valor (só na linha de comando do mongosh, em blocos e scripts é permitido apenas a reatribuição). No hoisting, variáveis declaradas com *let* NÃO recebem valor inicial (no mongosh, recebem *undefined* somente caso façamos uma declaração sem atribuir valor).
 - c) **const <nome-da-variável> = valor:** cria uma variável com escopo de bloco (existe apenas no bloco - {} - em que foi criada), a qual é possível redeclarar e reatribuir valor (só na linha de comando do mongosh, em blocos e scripts não é permitido nenhuma das duas ações). No hoisting, variáveis declaradas com *const* NÃO recebem valor inicial, sendo obrigatório inicializar a variável no momento da declaração.

Variável *db*: funciona como atalho para o banco de dados que estamos utilizando no momento.

- 5) ***db.colecao.insertOne(...)***: insere um documento na coleção especificada. Podemos inserir utilizando uma variável que contenha informações do documento ou escrevendo-o diretamente.
- 6) ***db.colecao.find(...)***: busca todos os documentos, na coleção especificada, que atendem um ou mais critérios estabelecidos. Se não recebe nenhum parâmetro, lista todos os documentos da coleção. Os critérios podem ser valores de campos do documento (sendo eles parciais, usando */<nome>/i*, ou inteiros) ou variáveis que contenham critérios específicos. Também podemos exibir documentos de forma parcial (Ex: *db.colecao.find({ }, { nome: 1, _id: 0 })*).

Query selectors:

- a) ***\$or***: utilizado dentro do método *.find()* para fazer buscas que atendam pelo menos um entre dois ou mais critérios estabelecidos (OU);
 - b) ***\$and***: utilizado dentro do método *.find()* para fazer buscas que atendam obrigatoriamente a dois ou mais critérios estabelecidos (E);
 - c) ***\$ne***: utilizado dentro do método *.find()*, em um campo específico, para buscar documentos que possuam valor diferente do estabelecido no critério para o campo em questão (Not Equal).
-
- 7) ***db.colecao.findOne(...)***: busca um único documento na coleção especificada, de acordo com um ou mais critérios estabelecidos (se houver mais de um, retorna o primeiro que encontrar). Se não recebe nenhum parâmetro, exibe o primeiro documento da coleção.
 - 8) ***db.colecao.countDocuments(...)***: retorna a contagem dos documentos da coleção especificada, de acordo com um ou mais critérios estabelecidos. Se não recebe nenhum parâmetro, conta todos os documentos da coleção.

9) ***db.colecao.remove(...)***: remove um documento da coleção de acordo com um ou mais critérios estabelecidos. Se não recebe nenhum parâmetro, exibe um erro. Se recebe um objeto vazio como parâmetro (`{}`), exclui todos os documentos da coleção. NOTA: Não é mais recomendada, use *.deleteOne()* para excluir apenas um documento que atenda os critérios especificados (se houver mais de um, deleta o primeiro encontrado) e *.deleteMany()* para excluir todos os documentos que atendam os critérios especificados (também deleta todos os documentos caso receba um objeto vazio).

10) ***db.colecao.update()***: busca e atualiza um documento da coleção de acordo com um ou mais critérios estabelecidos (se houver mais de um, atualiza apenas o primeiro que encontrar). Atualmente, também foi substituído por *.updateOne()*, *.updateMany()* e *.replaceOne()*. Podemos usar as atualizações para fazer embedded documents, que permitem buscar um documento sem a necessidade de realizar um join, o que torna as consultas mais rápidas. O método *.update()* recebe três parâmetros:

- a) **Primeiro parâmetro:** os critérios de busca;
- b) **Segundo parâmetro:** os campos a serem atualizados (devemos usar o seletor `$set` seguido dos novos valores nas versões atuais, que podem ser escritos diretamente ou em uma variável. Nas versão antiga, o `$set` não é obrigatório, mas sua ausência faz o documento anterior ser completamente substituído);
- c) **Terceiro parâmetro:** um objeto de opções, que define propriedades específicas do comportamento da atualização. Caso o documento não exista, por exemplo, podemos incluí-lo utilizando `{ upsert : true }` no terceiro parâmetro, e se quisermos que o método atualize todos os documentos que encontrar, utilizamos `{ multi : true }`.